КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМ. ІГОРЯ СІКОРСЬКОГО

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №4

на тему: Особливості реалізації програмних систем із криптографічним захистом

Виконали:
студентки групи ФІ-62 мн
Дегтярьова Марія
Тимошенко Марія

Київ – 2026

Зміст

1 Дослідження способів зберігання криптографічних ключів 2

2 Захист ключової інформації в оперативній пам'яті 8

3 Дослідження атак за побічними каналами 10

4 Аналіз ситуацій порушення безпеки 12

5 Проєктування системи керування ключами 15

6 Аналіз життєвого циклу ключів 17

Висновки 21

Перелік використаних джерел 22

Мета роботи: дослідження принципів безпечної реалізації та експлуатації програмних систем із криптографічним захистом, способів зберігання та використання криптографічних ключів, засобів апаратного захисту ключової інформації та основних загроз, пов'язаних із практичною реалізацією криптографічних механізмів.

1 Дослідження способів зберігання криптографічних ключів

1.1 Які загрози виникають при зберіганні криптографічних ключів безпосередньо у програмному коді, конфігураційних файлах або файловій системі?

Зберігання криптографічних ключів безпосередньо у програмному коді або звичайних файлах створює додаткові ризики для безпеки системи. Наприклад, ключ може випадково потрапити до відкритого репозиторію GitHub разом із вихідним кодом програми або стати доступним через неправильні налаштування прав доступу до файлів. Також небезпеку становлять резервні копії та журнали роботи програм, у яких може залишатися конфіденційна інформація.

У разі отримання ключа зловмисник може розшифрувати захищені дані, підробити цифровий підпис або отримати доступ до інших ресурсів системи. Крім того, якщо один ключ використовується у декількох компонентах програми, його компрометація може вплинути на всю систему. Тому ключі не рекомендується зберігати у відкритому вигляді в коді або конфігураційних файлах. Для цього краще використовувати спеціалізовані засоби керування секретною інформацією [1, 2].

1.2 Які можливості для захищеного зберігання ключів та іншої секретної інформації надають сучасні операційні системи та програмні платформи?

Сучасні операційні системи мають спеціальні механізми для зберігання паролів, криптографічних ключів та інших конфіденційних даних. Вони дозволяють обмежувати доступ до секретної інформації, використовувати шифрування та визначати, які програми можуть працювати з певними ключами.

Наприклад, у Windows використовуються Windows Credential Manager та DPAPI, у macOS — Keychain, а в Android — Android Keystore. Останній, зокрема, дозволяє створювати ключі, які не можуть бути безпосередньо отримані програмою. На пристроях із відповідною апаратною підтримкою їх використання може додатково захищатися спеціальними апаратними компонентами. Таким чином, операційна система не просто зберігає ключі, а й контролює умови їх використання, що зменшує ризик викрадення секретної інформації [3, 4, 5].

1.3 Що таке TPM (Trusted Platform Module) та для яких задач захисту криптографічних ключів він може використовуватися?

TPM (Trusted Platform Module) — це спеціальний апаратний криптографічний модуль, призначений для підвищення безпеки комп'ютера. Він може бути реалізований у вигляді окремого мікрочипа або інтегрованого компонента. TPM дозволяє генерувати та зберігати криптографічні ключі, а також обмежувати умови їх використання. При цьому приватний ключ може залишатися всередині модуля, не передаючись програмам у відкритому вигляді.

Одним із прикладів використання TPM є захист ключів шифрування диска за допомогою BitLocker у Windows. Також він застосовується для автентифікації пристроїв і перевірки цілісності процесу завантаження операційної системи. Наприклад, використання певного ключа можна дозволити лише тоді, коли система завантажилася у визначеному довіреному стані. Це ускладнює отримання доступу до зашифрованих даних у разі викрадення комп'ютера [6].

1.4 Що таке Secure Enclave та в чому полягає основна ідея апаратно ізольованого зберігання і використання ключів?

Secure Enclave — це спеціальне апаратно ізольоване середовище, яке використовується в пристроях Apple для захисту конфіденційної інформації та криптографічних ключів. Воно має окремий захищений процесор і працює незалежно від основної операційної системи.

Основна ідея полягає в тому, що приватний ключ створюється та зберігається всередині захищеного середовища і не передається основній системі у відкритому вигляді. Якщо програмі потрібно створити цифровий підпис або виконати іншу криптографічну операцію, вона надсилає відповідний запит до Secure Enclave, який самостійно виконує необхідні дії та повертає результат. Це зменшує ризик викрадення ключа навіть у випадку компрометації основної операційної системи. Такий механізм використовується, зокрема, для захисту ключів, пов'язаних з автентифікацією користувачів та цифровими підписами [7].

1.5 Що таке Hardware Security Module (HSM) та які задачі він вирішує?

Hardware Security Module (HSM) — це спеціалізований апаратний пристрій, призначений для створення, зберігання та використання криптографічних ключів. Його основна особливість полягає в тому, що криптографічні операції виконуються безпосередньо всередині захищеного модуля, тому приватні ключі не потрібно передавати до звичайної пам'яті комп'ютера.

HSM використовується для шифрування та розшифрування інформації, створення цифрових підписів, захисту ключів центрів сертифікації та обробки платіжних операцій. Такі пристрої особливо актуальні для банків, великих компаній та інформаційних систем, у яких обробляється значна кількість конфіденційних даних. Вони також можуть проходити спеціальну сертифікацію відповідно до стандарту FIPS 140-3, який визначає вимоги до безпеки криптографічних модулів [8, 9].

1.6 У чому полягає різниця між зберіганням криптографічного ключа у звичайному файлі, OS Keychain/Keystore, TPM, Secure Enclave та HSM? Для яких сценаріїв можуть бути доцільними ці підходи?

Основна різниця між розглянутими способами полягає у рівні ізоляції ключів та можливостях контролю доступу до них. Зберігання у звичайному файлі є найпростішим варіантом, проте він забезпечує найменший рівень захисту, оскільки безпека ключа значною мірою залежить від налаштувань доступу та додаткового шифрування.

OS Keychain/Keystore забезпечує системне сховище, у якому доступ до ключів контролюється операційною системою. Такий підхід підходить для звичайних програм і мобільних застосунків. TPM та Secure Enclave використовують апаратну ізоляцію, завдяки якій приватні ключі можуть залишатися всередині захищеного компонента. Відмінність полягає в тому, що TPM переважно використовується для захисту комп'ютерних систем, а Secure Enclave є рішенням для пристроїв Apple.

HSM, на відміну від попередніх механізмів, є спеціалізованим рішенням для організацій, яким необхідно централізовано керувати великою кількістю ключів та виконувати криптографічні операції з високими вимогами до безпеки.

Основні відмінності між цими підходами наведено в таблиці:

Табл. 1: Порівняння способів зберігання криптографічних ключів

Спосіб	**Особливості**	**Сфера використання**
Звичайний файл	Залежить від прав доступу та шифрування файлу	Навчальні проєкти, тестування
OS Keychain/Keystore	Системний контроль доступу та захищене зберігання	Мобільні та настільні застосунки
TPM	Апаратний захист ключів, прив'язка до пристрою	Захист комп'ютерів, шифрування дисків
Secure Enclave	Апаратна ізоляція ключів від основної системи	Пристрої Apple, автентифікація
HSM	Спеціалізований апаратний захист і виконання криптографічних операцій	Банки, платіжні системи, великі організації

Отже, вибір способу зберігання залежить від призначення системи та вимог до її безпеки. Для звичайних застосунків достатньо можливостей Keychain або Keystore, тоді як для захисту важливих системних ключів доцільно використовувати TPM, Secure Enclave або HSM. Водночас навіть апаратний захист не усуває всіх ризиків, тому важливо також контролювати доступ до ключів та своєчасно виконувати їх заміну у разі компрометації [1, 2].

1.7 Що означають поняття exportable та non-exportable key? У яких випадках доцільно забороняти експорт закритого ключа?

Поняття exportable key (експортований ключ) означає криптографічний ключ, який може бути витягнутий зі сховища ключів або криптографічного модуля у відкритому чи зашифрованому (wrapped) вигляді для перенесення в інше середовище, резервного копіювання або використання прикладним кодом. Натомість non-exportable key (неекспортований ключ) генерується безпосередньо всередині захищеного середовища (HSM, TPM, Secure Enclave або ізольованого сховища ОС) і не може покинути його межі через штатні інтерфейси ані у відкритому, ані в зашифрованому вигляді, оскільки всі криптографічні операції з ним виконуються виключно всередині цього модуля [1, 9].

Забороняти експорт закритого ключа доцільно у випадках, коли його копіювання чи компрометація призводить до критичних або незворотних наслідків для безпеки всієї системи. Насамперед це стосується ключів центрів сертифікації (Root CA та Intermediate CA), компрометація яких підриває довіру до всієї інфраструктури відкритих ключів, а також ключів створення кваліфікованого або юридично значущого електронного підпису, де критично важливо забезпечити принцип неспростовності (non-repudiation) [11]. Крім того, режим non-exportable застосовують для кореневих ключів шифрування (Master Keys) у платіжних системах, апаратних ключів автентифікації пристроїв та в середовищах, де навіть системний адміністратор сервера не повинен мати можливості скопіювати секрет [1].

1.8 Що таке PKCS#11 та яку проблему вирішує цей стандарт?

PKCS#11 (Cryptographic Token Interface Standard) — це відкритий стандарт, розроблений компанією RSA Laboratories і підтримуваний консорціумом OASIS. Він описує універсальний програмний інтерфейс Cryptoki (набір функцій мовою C), який дозволяє програмам взаємодіяти з різними криптографічними пристроями: апаратними модулями безпеки (HSM), смарткартками або USB-токенами [12].

Головна проблема, яку вирішує PKCS#11, — це несумісність обладнання від різних виробників. Раніше кожна компанія створювала власний закритий інтерфейс для своїх пристроїв, через що програмістам доводилося переписувати код під кожен конкретний токен чи HSM (виникала залежність від одного постачальника — vendor lock-in). Стандарт PKCS#11 вводить єдині правила роботи: програма звертається до пристрою через стандартні команди, а всі операції з ключами виконуються всередині самого носія без передачі

секретних даних у пам'ять комп'ютера [12].

1.9 Як програмний застосунок може виконати операцію цифрового підпису за допомогою HSM, не отримуючи значення закритого ключа?

Виконання операції цифрового підпису без вилучення закритого ключа відбувається шляхом делегування криптографічних обчислень самому апаратному модулю. Спочатку застосунок встановлює захищену сесію з HSM через стандартизований інтерфейс (наприклад, PKCS#11, CNG або JCA/JCE) та проходить процедуру автентифікації за допомогою PIN-коду, облікових даних криптокористувача (Crypto User) або клієнтського сертифіката. Після цього програма знаходить потрібний закритий ключ усередині модуля за його логічним ідентифікатором (Handle або Label), не отримуючи самого значення ключа [12, 8].

Далі застосунок обчислює криптографічний геш документа (наприклад, SHA-256) локально або передає вихідні дані до модуля для гешування і викликає функцію створення підпису (таку як `C_Sign` у PKCS#11), передаючи в HSM геш-значення та посилання на ключ. Захищений криптопроцесор HSM виконує математичну операцію підписання у своїй ізольованій пам'яті та повертає програмі лише готовий результат у вигляді цифрового підпису [12, 8].

1.10 Для чого використовується ротація криптографічних ключів? Які проблеми необхідно враховувати під час заміни ключа, яким уже зашифровано дані?

Ротація ключів (Key Rotation) — це процедура планової або позапланової заміни чинного криптографічного ключа на новий. Вона застосовується для обмеження обсягу даних, зашифрованих одним ключем, що знижує ймовірність успішного криптоаналізу, мінімізує потенційні збитки у разі невиявленої компрометації ключа, а також забезпечує відповідність вимогам галузевих стандартів безпеки [11, 1].

Під час заміни ключа, яким уже зашифровано значні масиви інформації, необхідно враховувати низку технічних проблем. По-перше, якщо старий ключ знищити одразу після генерації нового, раніше зашифровані записи стануть назавжди недоступними. По-друге, негайне розшифрування всього масиву даних старим ключем і повторне шифрування новим може створити критичне навантаження на процесор і спричинити блокування бази даних. По-третє, система повинна підтримувати версіонування ключів, зберігаючи ідентифікатор версії ключа разом із кожним зашифрованим блоком. Для розв'язання цих проблем використовують конвертне шифрування (Envelope Encryption), за якого дані шифруються ключем даних (DEK), а сам DEK — головним ключем (KEK), що дозволяє під час ротації перешифровувати лише невеликі ключі DEK замість усього обсягу да-

них [1, 11].

1.11 У яких випадках необхідне резервне копіювання ключової інформації та які додаткові ризики при цьому виникають?

Резервне копіювання ключової інформації є необхідним для ключів шифрування даних (Data Encryption Keys та Master Keys), оскільки їхня втрата через апаратний збій, стихійне лихо або помилку адміністратора призведе до остаточної неможливості відновити критично важливу інформацію. Натомість ключі цифрового підпису зазвичай не підлягають резервному копіюванню — у разі їх втрати генерується нова ключова пара та випускається новий сертифікат [11, 1].

Водночас створення резервних копій ключів породжує додаткові ризики безпеки. Кожна створена копія підвищує ймовірність компрометації та викрадення, а також ускладнює контроль життєвого циклу, оскільки резервні копії можуть роками зберігатися на зовнішніх носіях навіть після виведення робочого ключа з експлуатації. Крім того, існує ризик несанкціонованого відновлення, якщо архівна копія захищена слабшим механізмом, ніж основне сховище. З огляду на це резервні копії ключів повинні зберігатися виключно у зашифрованому вигляді (Key Wrapping) із застосуванням механізмів розділення повноважень, таких як схема поділу секрету Шаміра [11].

1.12 Які стандарти регулюють вимоги до безпеки реалізації криптографічних модулів?

Вимоги до проєктування, реалізації та перевірки захищеності криптографічних модулів регламентуються низкою міжнародних і державних стандартів. Ключовим галузевим стандартом є FIPS 140-3 (розроблений NIST), який визначає чотири рівні безпеки (Security Levels 1–4) для апаратних і програмних криптографічних модулів, включно з вимогами щодо захисту від фізичного втручання та атак за побічними каналами, базуючись на міжнародному стандарті ISO/IEC 19790 [9].

Для всебічного оцінювання захищеності інформаційних технологій та сертифікації смарткарток, TPM і HSM застосовується міжнародний стандарт Common Criteria (ISO/IEC 15408), який використовує профілі захисту та рівні гарантій оцінювання (EAL1–EAL7) [13]. Організаційні та технічні аспекти керування ключами на всіх етапах їхнього існування детально регламентуються рекомендаціями NIST SP 800-57 [11], тоді як у фінансовому та платіжному секторі додатково діють спеціалізовані вимоги стандарту PCI PTS HSM (PIN Transaction Security — Hardware Security Module) [14].

2 Захист ключової інформації в оперативній пам'яті

2.1 Чому ключ, який надійно зберігається у зашифрованому файлі, може стати доступним у відкритому вигляді під час роботи програми?

Навіть якщо криптографічний ключ зберігається у зашифрованому вигляді, під час роботи програми його все одно потрібно розшифрувати для виконання криптографічних операцій. У результаті відкритий ключ певний час перебуває в оперативній пам'яті комп'ютера, де його потенційно можуть отримати сторонні особи.

Наприклад, якщо зловмисник отримає доступ до процесу програми або зможе виконати шкідливий код на комп'ютері, він може спробувати прочитати ділянки пам'яті, у яких тимчасово зберігається ключ. Також ризик виникає через використання налагоджувачів або спеціальних засобів аналізу пам'яті. Тому шифрування файлу захищає ключ під час зберігання, але не гарантує його безпеки під час використання. Для зменшення ризиків рекомендується обмежувати час перебування секретних даних у пам'яті та, за можливості, використовувати захищені апаратні механізми, які дозволяють виконувати криптографічні операції без розкриття приватного ключа [1, 10].

2.2 Які ризики для ключової інформації створюють memory dump, core dump, crash dump та swap?

Під час роботи програми криптографічні ключі та інші секретні дані можуть тимчасово зберігатися в оперативній пам'яті. Проблема полягає в тому, що деякі системні механізми дозволяють зберігати вміст пам'яті на диску, через що конфіденційна інформація може залишитися доступною навіть після завершення роботи програми.

Memory dump — це копія вмісту оперативної пам'яті, яка може містити криптографічні ключі, паролі та інші секретні дані. Core dump і crash dump створюються переважно під час аварійного завершення роботи програми або системи та використовуються для пошуку помилок. Проте разом із технічною інформацією вони можуть зберігати й конфіденційні дані з пам'яті процесу. Swap, у свою чергу, використовується операційною системою для тимчасового перенесення частини даних з оперативної пам'яті на диск. Якщо секретний ключ потрапить до swap-файлу або розділу, він може залишитися там навіть після завершення роботи програми.

Таким чином, основна небезпека полягає в тому, що секретна інформація може зберігатися у незахищеному вигляді поза межами оперативної пам'яті. Для зменшення таких ризиків необхідно обмежувати створення дампів, захищати файли підкачки та контролювати доступ до діагностичних файлів [10, 1].

2.3 Для чого після використання секретних даних може виконуватися очищення відповідних ділянок пам'яті?

Після завершення роботи із секретними даними рекомендується очищати ділянки оперативної пам'яті, у яких вони зберігалися. Це необхідно для того, щоб ключі, паролі та інша конфіденційна інформація не залишалися в пам'яті довше, ніж це потрібно програмі. Якщо такі дані не очищати, зловмисник потенційно може отримати до них доступ через аналіз пам'яті процесу або створення її дампа.

Особливо важливо очищати пам'ять після використання приватних криптографічних ключів, оскільки їх викрадення може призвести до розшифрування даних або підробки цифрових підписів. При цьому звичайного присвоєння змінній порожнього значення не завжди достатньо, оскільки компілятор або середовище виконання можуть оптимізувати таку операцію. Тому для очищення слід використовувати спеціальні функції безпечного обнулення пам'яті, які гарантують виконання запису. Це зменшує час, протягом якого секретна інформація залишається доступною, хоча й не забезпечує абсолютного захисту від усіх способів її отримання [10, 1].

2.4 Чому гарантоване видалення секретної інформації з оперативної пам'яті може бути складнішим у мовах із автоматичним керуванням пам'яттю?

У мовах програмування з автоматичним керуванням пам'яттю (таких як Java, C#, Python, Go або JavaScript) розробник не контролює фізичне виділення та звільнення адресного простору, що суттєво ускладнює надійне очищення секретних даних. Насамперед це пов'язано з недетермінованістю роботи збирача сміття (Garbage Collector): після втрати посилання на об'єкт ключ або пароль залишається в оперативній пам'яті невизначений час, доки система не вирішить звільнити та перевикористати цю ділянку. Крім того, під час компактифікації купи (heap) збирач сміття може неявно переміщувати живі об'єкти в іншу область пам'яті, залишаючи в старій ділянці незатерту копію секрету, про існування якої програма навіть не знає [2, 15].

Додатковою проблемою є незмінність рядків (immutability) у більшості високорівневих мов, через що будь які операції з об'єктами типу String створюють нові копії в пам'яті без можливості перезаписати старий буфер. Навіть якщо програма явно перезаписує масив байтів нулями, оптимізуючий JIT-компілятор може видалити цю інструкцію під час виконання (Dead Store Elimination), якщо виявить, що до змінної більше немає звернень. Тому в таких середовищах для роботи із секретами рекомендується використовувати змінні масиви байтів замість рядків або спеціалізовані механізми на кшталт SecureString, ProtectedMemory чи виділення закріпленої некерованої пам'яті (pinned / unmanaged memory) [2, 15].

2.5 Чим з погляду захисту ключа відрізняється виконання криптографічної операції програмою та виконання цієї операції всередині HSM або іншого захищеного апаратного середовища?

Головна відмінність між програмним та апаратним виконанням криптографічної операції полягає у межах довіри та фізичній локалізації закритого ключа під час обчислень. При програмному виконанні криптографічний ключ має бути завантажений у звичайну оперативну пам'ять (RAM) центрального процесора хост-системи у відкритому вигляді. Це робить його вразливим до компрометації операційної системи, зчитування пам'яті привілейованими процесами чи налагоджувачами, збереження у дампах пам'яті (crash dumps) або файлі підкачки (swap), а також до атак через спільний кеш процесора [15, 1].

Натомість під час виконання операції всередині HSM, TPM або Secure Enclave закритий ключ ніколи не покидає ізольовану внутрішню пам'ять захищеного криптопроцесора і не передається в оперативну пам'ять основного комп'ютера. Операційна система та прикладне програмне забезпечення не мають прямого доступу до адресного простору цього модуля, а взаємодіють із ним лише шляхом передачі вхідних даних і отримання готового результату. Крім логічної ізоляції, захищене апаратне середовище забезпечує фізичний захист від втручання (tamper-response з миттєвим стиранням ключів при спробі розкриття корпусу) та апаратні механізми протидії атакам за побічними каналами [8, 9].

3 Дослідження атак за побічними каналами

Атаки за побічними каналами (side-channel attacks) — це методи отримання секретної інформації шляхом аналізу особливостей роботи криптографічної системи, а не безпосереднього зламу математичних властивостей алгоритму. Під час виконання криптографічних операцій пристрій може ненавмисно розкривати певну інформацію через час виконання операцій, споживання електроенергії, електромагнітне випромінювання або особливості використання кеш-пам'яті. Аналізуючи такі характеристики, атакуючий може отримати відомості про секретний ключ або проміжні результати обчислень.

Одним із видів таких атак є timing attacks (атаки за часом виконання). Їхня суть полягає у вимірюванні часу, необхідного для виконання певної операції. Якщо тривалість роботи програми залежить від значень секретних даних, атакуючий може аналізувати ці відмінності та поступово отримувати інформацію про них. Наприклад, якщо перевірка пароля або криптографічного значення припиняється одразу після виявлення першої невідповідності, час виконання може залежати від кількості правильно вгаданих символів.

Cache attacks (атаки на кеш) використовують особливості роботи кеш-пам'яті процесора. Оскільки доступ до даних, які вже знаходяться в кеші, зазвичай швидший, ніж до даних з оперативної пам'яті, за часом доступу до певних ділянок пам'яті можна ро-

бити висновки про дії програми. Таким чином, навіть якщо атакуючий не має прямого доступу до секретних даних, він може отримати інформацію про їх використання під час обчислень.

Power analysis (аналіз енергоспоживання) ґрунтується на дослідженні кількості електроенергії, яку споживає пристрій під час виконання криптографічних операцій. Споживання енергії може змінюватися залежно від оброблюваних даних та виконуваних інструкцій. Аналізуючи такі зміни, атакуючий може встановити залежності між проміжними обчисленнями та секретним ключем.

Electromagnetic analysis (електромагнітний аналіз) передбачає дослідження електромагнітного випромінювання, яке виникає під час роботи електронного пристрою. Таке випромінювання може містити інформацію про виконувані операції та оброблювані дані. За допомогою спеціального обладнання атакуючий може фіксувати ці сигнали й аналізувати їх для отримання відомостей про секретну інформацію.

Особливість атак за побічними каналами полягає в тому, що вони не обов'язково порушують математичну стійкість самого криптографічного алгоритму. Наприклад, алгоритм може мати достатню довжину ключа та бути стійким до відомих методів криптоаналізу, але його програмна реалізація може виконувати різну кількість операцій залежно від значення секретного ключа. Аналогічно апаратна реалізація може створювати помітні відмінності в енергоспоживанні або електромагнітному випромінюванні. Тому під час розробки криптографічних систем важливо враховувати не лише надійність алгоритму, а й особливості його практичної реалізації.

Розглянемо приклад перевірки правильності MAC (Message Authentication Code), яка виконується за допомогою такого фрагмента коду:

```
for i in range(len(expected_mac)):
    if received_mac[i] != expected_mac[i]:
        return False
return True
```

У цьому прикладі порівняння виконується посимвольно, а у випадку першої невідповідності функція одразу завершує роботу та повертає значення `False`. Якщо ж усі символи збігаються, перевірка продовжується до кінця.

Таким чином, час виконання алгоритму залежить від того, на якій позиції виявлено першу невідповідність. Якщо перші кілька символів отриманого MAC збігаються з правильним значенням, програма виконає більше ітерацій. Якщо ж невідповідність виявиться на початку, функція завершиться значно швидше. Отже, час перевірки потенційно залежить від значення отриманого MAC.

Вимірюючи час виконання великої кількості таких перевірок, атакуючий може поступово отримувати інформацію про правильне значення MAC. Зокрема, він може визначати, які початкові символи або байти збігаються з очікуваним значенням. Повторюючи такі спроби, можна поступово відновлювати правильний MAC, навіть не маючи безпосереднього доступу до секретного ключа. На практиці успішність такої атаки залежить від

можливості достатньо точно вимірювати час виконання та від наявності інших джерел шуму.

Для запобігання подібним атакам використовують constant-time comparison — порівняння за сталим часом. Його основна ідея полягає в тому, щоб тривалість перевірки не залежала від позиції першої невідповідності або від значень секретних даних. Тобто порівняння має виконуватися за однаковим алгоритмом незалежно від того, чи збігаються значення повністю, частково або не збігаються взагалі.

Самостійно реалізовувати таке порівняння не завжди доцільно, оскільки компілятор або особливості процесора можуть впливати на час виконання операцій. Тому для перевірки MAC, криптографічних токенів та інших секретних значень рекомендується використовувати спеціалізовані функції криптографічних бібліотек. Такі функції розробляються з урахуванням ризиків атак за часом виконання та призначені для зменшення залежності тривалості перевірки від оброблюваних секретних даних.

Отже, захист криптографічної системи залежить не лише від вибору стійкого алгоритму, а й від того, наскільки безпечно організовано його програмну та апаратну реалізацію. Навіть незначні відмінності в часі виконання або інших характеристиках роботи пристрою можуть стати джерелом витоку конфіденційної інформації.

4 Аналіз ситуацій порушення безпеки

4.1 Ситуація 1. Ключ у програмному коді

Опис ситуації: Розробник зберігає ключ шифрування безпосередньо у вихідному коді програми, який розміщений у Git-репозиторії.

Проблема: Такий підхід є небезпечним, оскільки вихідний код можуть переглядати інші розробники або особи, які отримали доступ до репозиторію. Навіть якщо ключ згодом видалити з коду, він може залишитися в історії комітів. Таким чином, секретна інформація фактично перестає бути секретною.

Можливі наслідки: У разі витоку ключа зловмисник може отримати доступ до зашифрованих даних, розшифрувати їх або використовувати ключ для створення підроблених повідомлень. Крім того, якщо один і той самий ключ використовується в різних частинах системи, компрометація одного репозиторію може створити загрозу для всіх пов'язаних сервісів.

Спосіб усунення: Ключі не слід зберігати безпосередньо у вихідному коді. Замість цього їх потрібно передавати через змінні середовища або використовувати спеціалізовані засоби керування секретами, наприклад HashiCorp Vault чи хмарні сховища ключів. Також варто обмежувати доступ до ключів, регулярно їх змінювати та не допускати потрапляння секретної інформації до Git-репозиторію.

4.2 Ситуація 2. Ключ поряд із зашифрованими даними

Опис ситуації: База даних сервера зашифрована за допомогою алгоритму AES, однак master key, необхідний для її розшифрування, зберігається у конфігураційному файлі на тому самому сервері.

Проблема: Основна проблема полягає в тому, що зашифровані дані та ключ для їх розшифрування знаходяться в одному місці. Це зменшує ефективність такого захисту у випадку, коли зловмисник отримує повний доступ до сервера. Водночас саме по собі зберігання ключа на сервері не робить шифрування марним, оскільки воно може захистити дані від осіб, які отримали лише копію бази даних.

Можливі наслідки: Якщо зловмисник отримає доступ тільки до файлів бази даних, він не зможе прочитати їхній вміст без ключа. Проте у випадку компрометації всього сервера він може отримати як зашифровану базу, так і master key. У такому разі дані можна буде повністю розшифрувати. Подібна ситуація можлива також у разі викрадення резервної копії сервера разом із конфігураційними файлами.

Спосіб усунення: Для підвищення безпеки ключ потрібно зберігати окремо від зашифрованих даних, наприклад у спеціалізованому сховищі ключів або апаратному модулі безпеки (HSM). Доступ до нього має бути обмежений і надаватися лише тим компонентам системи, яким він дійсно необхідний. Також доцільно використовувати механізм керування ключами (KMS), який дозволяє контролювати доступ до ключів та здійснювати їх ротацію.

4.3 Ситуація 3. Розголошення секретної інформації через журнали

Опис ситуації: Для діагностики помилок застосунок записує до журналів значення криптографічних ключів, access tokens або незашифровані дані, які використовуються перед виконанням криптографічних операцій.

Проблема: Проблема полягає в тому, що журнали можуть містити конфіденційну інформацію, яка не повинна бути доступною стороннім особам. При цьому до журналів часто мають доступ адміністратори, розробники або інші служби, яким не обов'язково знати секретні дані. Крім того, журнали можуть зберігатися тривалий час і копіюватися на інші сервери.

Можливі наслідки: У разі витоку журналів зловмисник може отримати криптографічні ключі або токени доступу, що дозволить йому розшифровувати дані, отримувати доступ до облікових записів чи виконувати операції від імені користувача. Якщо в журналах містяться незашифровані персональні або конфіденційні дані, це також може призвести до порушення їхньої конфіденційності.

Спосіб усунення: Необхідно заборонити запис криптографічних ключів, паролів, токенів доступу та інших секретних даних до журналів. Для діагностики варто використовувати ідентифікатори операцій, коди помилок та іншу інформацію, яка не розкриває

конфіденційних відомостей. Якщо певні дані необхідно відображати для налагодження, їх слід попередньо маскувати або знеособлювати. Також потрібно обмежувати доступ до журналів, захищати їх від несанкціонованої зміни та видалення, а також встановлювати обґрунтовані строки їх зберігання.

4.4 Ситуація 4. Захист ключа цифрового підпису

Опис ситуації: Серверний сервіс повинен створювати цифрові підписи документів. При цьому навіть системний адміністратор сервера не повинен мати можливості отримати копію закритого ключа.

Проблема: Якщо закритий ключ зберігається у файловій системі сервера або завантажується у звичайну оперативну пам'ять процесу у відкритому вигляді, системний адміністратор зі своїми правами може прочитати файл ключа, зняти дамп пам'яті процесу або скористатися налагоджувачем для вилучення секрету.

Можливі наслідки: У разі зловмисних дій інсайдера або захоплення облікового запису адміністратора зловмисник зможе скопіювати закритий ключ і підробляти цифрові підписи документів від імені сервісу поза межами системи, що руйнує їхню юридичну значущість та принцип неспростовності.

Спосіб усунення: Для зберігання та використання ключа слід застосувати апаратний модуль безпеки (HSM), усередині якого генерується закритий ключ із атрибутом non-exportable. Роль HSM полягає в апаратній ізоляції ключа та виконанні криптографічних операцій безпосередньо всередині захищеного криптопроцесора, тоді як прапорець non-exportable фізично унеможливлює експорт закритого ключа назовні навіть адміністратором. Взаємодія серверного сервісу з модулем відбувається через стандартизований інтерфейс PKCS#11 (або аналогічний API): сервіс передає в HSM лише геш документа та отримує готовий підпис, не маючи доступу до самого ключа.

4.5 Ситуація 5. Побічний канал

Опис ситуації: Криптографічна бібліотека правильно реалізує алгоритм HMAC, але програма порівнює отриманий MAC зі справжнім звичайною функцією порівняння.

Проблема: Стандартні функції порівняння працюють за принципом раннього виходу: вони перевіряють дані побайтово і зупиняються одразу після виявлення першої розбіжності. Через це час виконання перевірки залежить від того, скільки початкових байтів у переданому MAC збігаються з правильним значенням.

Можливі наслідки: Система стає вразливою до атаки за часом (timing attack). Зловмисник може надсилати серію запитів із різними варіантами MAC, вимірювати час відповіді сервера та за статистичною різницею поступово побайтово відновити коректний код автентифікації для підробленого повідомлення.

Спосіб усунення: Необхідно замінити звичайне порівняння на спеціалізовану функцію порівняння за сталий час (constant-time comparison) із перевіреної криптографічної

бібліотеки, наприклад `hmac.compare_digest()` у Python або `CRYPTO_memcmp()` в OpenSSL. Такі функції завжди перевіряють усі байти обох значень за фіксований час незалежно від позиції, на якій виникла невідповідність.

4.6 Ситуація 6. Компрометація закритого ключа

Опис ситуації: Закритий production key випадково потрапив до публічного Git-репозиторію. Через декілька хвилин файл було видалено.

Проблема: Видалення файла у новому коміті не усуває проблему, оскільки файл залишається доступним в історії комітів Git. Крім того, публічні репозиторії постійно скануються автоматизованими ботами зловмисників у реальному часі, тому навіть кількох хвилин достатньо для копіювання секрету.

Можливі наслідки: Подальше використання цього ключа категорично заборонено, оскільки його необхідно вважати безумовно скомпрометованим. Інакше зловмисники зможуть розшифровувати конфіденційні дані, підробляти цифрові підписи, видавати себе за легітимний сервер або отримувати несанкціонований доступ до систем.

Спосіб усунення: Після виявлення компрометації необхідно виконати комплекс невідкладних дій: припинити використання скомпрометованого ключа, випустити новий ключ, переналаштувати сервіси, повністю очистити історію комітів Git від секретного файла та провести аудит журналів доступу за період інциденту. Ключову роль у цьому процесі відіграють три механізми: відкликання (revocation) негайно анулює довіру до старого сертифіката чи облікових даних через списки відкликання (CRL/OCSP), заміна ключа (key replacement) забезпечує екстрену генерацію нової ключової пари замість скомпрометованої, а ротація (key rotation) дозволяє безпечно оновити ключі шифрування та перешифрувати захищені ними дані новим ключем без втрати доступу до інформації.

5 Проєктування системи керування ключами

Для забезпечення конфіденційності, цілісності та автентичності даних в інформаційній системі, що складається з backend-сервісу та бази даних, визначено основні типи криптографічних ключів та секретної інформації. Їхні характеристики наведено в таблиці 2.

Секрет	**Призначення**	**Зберігання**	**Хто має доступ**	**Експорт**	**Ротація**
Ключ шифрування БД	Шифрування та розшифрування конфіденційних даних користувачів	Захищене сховище ключів (KMS) або HSM	Backend-сервіс через спеціальний механізм доступу	Ні, бажано не експортувати у відкритому вигляді	Періодична заміна зі збереженням можливості розшифрування старих даних
Ключ цифрового підпису	Створення цифрових підписів документів та підтвердження їх автентичності	HSM або захищене сховище ключів	Backend-сервіс, який виконує підписання документів	Ні, приватний ключ не повинен експортуватися	Періодична заміна ключа; старі ключі зберігаються для перевірки раніше створених підписів
TLS private key	Встановлення захищеного з'єднання (HTTPS/TLS) та автентифікація сервера	Сховище ключів ОС або захищена директорія з обмеженими правами	Веб-сервер / Reverse Proxy (або Backend-сервіс)	Бажано заборонити (допустимо лише для балансування)	Регулярна автоматична заміна (кожні 60–90 днів) із перевипуском сертифіката
API credential	Автентифікація при зверненні до зовнішніх API-сервісів	Менеджер секретів або захищені змінні середовища процесу	Лише модуль інтеграції у складі Backend-сервісу	Так, передається під час автентифікації у зовнішньому API	Періодична заміна (кожні 30–90 днів) із періодом паралельної дії

Табл. 2: Характеристики різних типів секретної інформації

Оскільки в разі інцидентів безпеки реагування залежить від типу секрету, для кожного з них визначено регламент дій у випадку компрометації. При компрометації ключа шифрування БД необхідно негайно згенерувати новий ключ, перешифрувати всі конфіденційні записи в базі даних, знищити скомпрометований ключ та провести аудит журналів доступу. У разі витоку ключа цифрового підпису потрібно терміново відкликати відповідний сертифікат через списки відкликання (CRL або OCSP), згенерувати нову ключову пару та перевірити всі документи, підписані в період потенційної компрометації. Якщо скомпрометовано TLS private key, слід негайно відкликати TLS-сертифікат у центрі сертифікації, створити нову пару ключів та встановити новий сертифікат на сервері. При компрометації API credential необхідно миттєво анулювати старий токен або ключ у панелі керування зовнішнього сервісу, випустити нові облікові дані, оновити налаштування backend-сервісу та перевірити історію операцій у зовнішньому API.

Спрощену архітектурну схему розміщення основних компонентів інформаційної системи та сховищ ключової інформації наведено на рисунку 1.

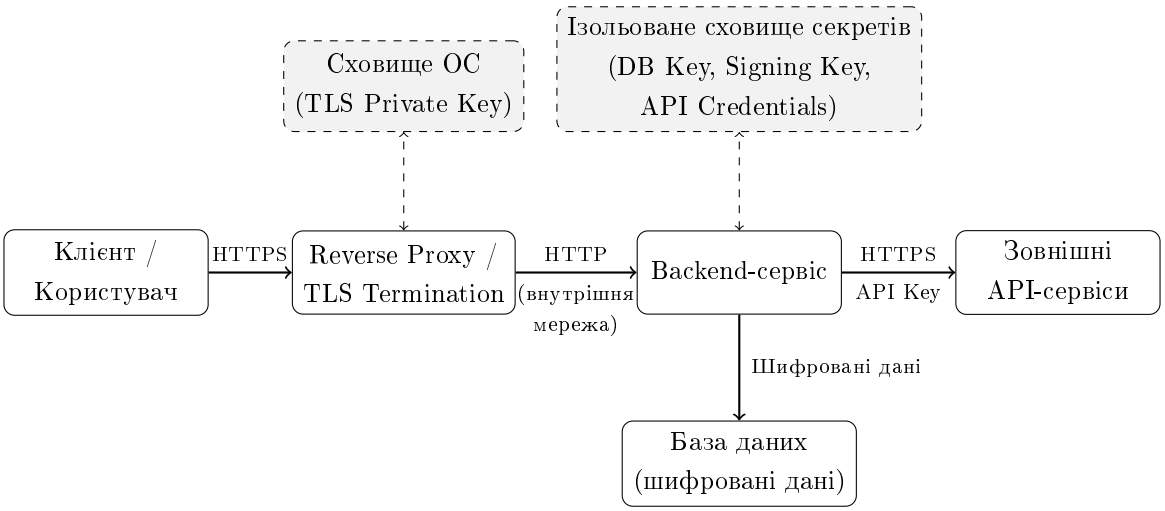


Рис. 1: Спрощена схема розміщення компонентів системи та сховищ ключів

Зберігання всіх криптографічних ключів і секретів безпосередньо разом із програмним застосунком (у вихідному коді, конфігураційних файлах чи локальній директорії проєкту) не є оптимальною архітектурою з кількох причин. По-перше, це порушує принцип розділення обов'язків (Separation of Duties): розробники та системи збірки, які мають доступ до коду застосунку, автоматично отримують доступ до секретів робочого середовища. По-друге, зростає ризик випадкового витоку конфіденційної інформації через системи контролю версій (Git), журнали налагодження або резервні копії сервера. По-третє, ускладнюється ротація та екстрена заміна скомпрометованих ключів, оскільки кожне оновлення секрету вимагає зміни конфігурації, повторного розгортання або перезапуску самого застосунку. Нарешті, розміщення ключів у спільній файловій системі із застосунком позбавляє систему централізованого аудиту звернень до секретів та апаратної ізоляції, через що будь-яка вразливість у вебзастосунку (наприклад, читання локальних файлів) призводить до миттєвої компрометації всіх криптографічних механізмів системи.

6 Аналіз життєвого циклу ключів

Для аналізу життєвого циклу обрано ключ шифрування бази даних та ключ цифрового підпису документів.

6.1 Життєвий цикл ключа шифрування бази даних

1. Генерація. Основними загрозами на етапі генерації є використання криптографічно слабкого генератора псевдовипадкових чисел, що робить ключ передбачуваним, а також перехоплення його значення під час створення чи запис у тимчасові файли й логи. Доступ до цієї операції повинні мати виключно автоматизований сервіс керування ключами або уповноважений адміністратор безпеки під час ініціалізації сховища. Надійність генерації забезпечується використанням криптографічно стійкого генератора випадкових чисел (CSPRNG) для формування ключа необхідної довжини (256 біт для алгоритму AES)

безпосередньо всередині захищеного середовища без виведення значення на екран чи в журнали. У випадку компрометації на етапі генерації необхідно негайно знищити скомпрометований екземпляр ключа, провести аудит середовища створення та згенерувати новий ключ.

2. Зберігання. На етапі зберігання головними загрозами є несанкціоноване читання файла ключа з диска, викрадення ключа разом із резервними копіями сервера або розміщення ключа в одному середовищі із зашифрованою базою даних. Доступ на цьому етапі має лише ізольоване сховище секретів (KMS або захищене сховище ОС), тоді як адміністратори бази даних не повинні мати прямого доступу до читання ключа. Захист забезпечується шляхом шифрування ключа головним ключем (Key Wrapping), суворого розмежування прав доступу (ACL) та відокремлення сховища ключів від сервера бази даних. У разі компрометації сховища потрібно негайно заблокувати облікові записи та токени доступу, провести аудит подій, виконати екстрену заміну ключа та перешифрувати базу даних.

3. Використання. Під час використання виникають ризики зчитування відкритого ключа з оперативної пам'яті процесу (через memory dump або crash dump), потрапляння ключа в область підкачки (swap) чи випадкового логування під час обробки помилок. Доступ до ключа на цьому етапі повинен мати виключно автентифікований процес backend-сервісу в момент виконання операцій шифрування або розшифрування конфіденційних полів. Для захисту необхідно мінімізувати час перебування відкритого ключа в оперативній пам'яті, заборонити вивантаження сторінок пам'яті у swap, примусово обнуляти буфери пам'яті після завершення криптографічної операції та фільтрувати журнали застосунку. При виявленні компрометації слід негайно зупинити скомпрометований екземпляр backend-сервісу, відкликати його облікові дані доступу до сховища, згенерувати новий ключ і перешифрувати захищені записи в базі даних.

4. Ротація. Основними загрозами при ротації є втрата доступу до раніше зашифрованих даних через передчасне видалення попереднього ключа, порушення цілісності інформації або критичне навантаження на базу даних під час масового перешифрування. Доступ до процесу ротації мають служба керування ключами та фоновий процес міграції даних у складі backend-сервісу. Захист забезпечується шляхом версіонування ключів (збереження ідентифікатора версії ключа в кожному записі БД), використання конвертного шифрування (Envelope Encryption) та переведення старого ключа в режим лише для розшифрування (decrypt-only) на період міграції. У випадку компрометації старого ключа під час ротації необхідно скасувати поступову фонову міграцію і запустити термінове примусове перешифрування всіх записів новим ключем із подальшим негайним знищенням скомпрометованого ключа.

5. Відкликання та знищення. На завершальному етапі головною загрозою є збереження залишкових копій ключа на дискових носіях, в оперативній пам'яті або в архівних резервних копіях, що дозволяє відновити ключ після завершення терміну його дії. Виконувати цю операцію має право лише адміністратор безпеки або автоматизова-

ний регламентний процес завершення життєвого циклу в сховищі ключів. Для надійного захисту застосовують криптографічне стирання (crypto-shredding — знищення головного ключа, яким зашифровано ключ даних), примусовий перезапис ділянок пам'яті нулями та видалення застарілого ключа з усіх кешів і резервних копій. У разі компрометації необхідно негайно видалити ключ з усіх вузлів системи та перевірити відсутність у базі даних записів, які залишилися зашифрованими цим ключем.

6.2 Життєвий цикл ключа цифрового підпису

1. Генерація. Основними загрозами на етапі генерації є створення криптографічно слабкої ключової пари або перехоплення закритого ключа в момент його формування за межами захищеного апаратного середовища. Право на ініціацію генерації має лише уповноважений адміністратор криптографічного модуля (Crypto Officer). Захист забезпечується генерацією ключової пари безпосередньо всередині HSM із встановленням атрибута non-exportable та формуванням запиту на сертифікат (CSR) для центру сертифікації. У випадку компрометації під час генерації необхідно негайно виконати апаратне видалення скомпрометованої ключової пари всередині модуля та згенерувати нову пару.

2. Зберігання. На етапі зберігання головними загрозами є фізичне втручання в роботу апаратного модуля, несанкціонований адміністративний доступ або спроби копіювання закритого ключа адміністраторами з розширеними правами доступу. Закритий ключ зберігається виключно в ізольованій внутрішній пам'яті HSM, а прямий доступ для його читання відсутній у всіх користувачів і адміністраторів системи. Захист реалізується через апаратну ізоляцію з прапорцем non-exportable, захист корпусу пристрою від фізичного розкриття (tamper-response) та багатофакторну автентифікацію адміністраторів за схемою розділення повноважень (M-of-N). У разі компрометації виконується апаратне занулення пам'яті модуля (zeroization) та негайне відкликання сертифіката відкритого ключа в центрі сертифікації.

3. Використання. Під час використання основними загрозами є несанкціоноване підписання підроблених документів у разі компрометації backend-сервісу, а також атаки за побічними каналами (наприклад, аналіз енергоспоживання чи часу обчислень) під час формування підпису. Доступ на цьому етапі надається лише backend-сервісу, якому дозволено викликати операцію підпису через стандартизований інтерфейс (наприклад, PKCS#11) без вилучення самого ключа. Для захисту застосовується сувора автентифікація застосунку перед HSM, обов'язкове журналювання кожної операції підписання та використання сертифікованих модулів із захистом від атак за побічними каналами. При виявленні компрометації необхідно негайно заблокувати обліковий запис backend-сервісу в HSM, відкликати чинний сертифікат і провести аудит журналу підписаних документів для виявлення фальсифікацій.

4. Ротація. Головними загрозами при ротації ключа підпису є втрата можливості перевірити автентичність раніше підписаних документів після заміни ключа або продовження використання старого закритого ключа для нових підписів. Доступ до процесу ротації

мають адміністратор криптографічного модуля та центр сертифікації (CA). Захист забезпечується генерацією нової ключової пари в HSM і випуском нового сертифіката, тоді як старий закритий ключ блокується для створення нових підписів, а відповідний відкритий ключ (сертифікат) і позначки часу (timestamps) зберігаються в архіві для верифікації раніше підписаних документів. У випадку компрометації виконується примусове позачергове відкликання старого сертифіката із зазначенням факту та точного часу компрометації ключа.

5. Відкликання та знищення. На завершальному етапі основними загрозами є використання простроченого чи скомпрометованого закритого ключа для створення нових підписів, а також відсутність актуальної інформації про анулювання сертифіката у перевіряючих сторін. Суб'єктами доступу на цьому етапі є адміністратор криптографічного модуля та центр сертифікації. Надійне завершення життєвого циклу забезпечується апаратним затиранням нулями закритого ключа всередині HSM (zeroization) після закінчення строку дії сертифіката та публікацією відомостей про дострокове припинення дії у списках відкликаних сертифікатів (CRL) і через протокол OCSP. У разі компрометації необхідно миттєво внести сертифікат до CRL/OCSP, апаратно знищити закритий ключ у HSM та сповістити всі зацікавлені сторони про припинення довіри до цього ключа.

6.3 Відмінність між плановою ротацією ключа та його екстреною заміною після компрометації

На відміну від планової ротації, яка проводиться за затвердженим графіком у штатному режимі, екстрена заміна ключа запускається через інцидент безпеки й вимагає негайного припинення довіри до старого ключа.

Планова ротація є регламентним запобіжним заходом, який виконується заздалегідь (наприклад, раз на рік або після шифрування граничного обсягу даних) з метою обмеження обсягу інформації, захищеної одним ключем, і зниження ризиків невиявленого витоку. Під час планової ротації попередній ключ залишається довіреним до моменту завершення свого криптоперіоду. Цифрові підписи, створені ним раніше, зберігають юридичну силу, а перешифрування даних у базі може виконуватися поступово у фоновому режимі без зупинки роботи системи.

Натомість екстрена заміна ключа є реакцією на підтверджений або ймовірний інцидент безпеки (витік ключа в репозиторій, злам сервера чи сховища секретів) і виконується негайно. У цьому випадку старий ключ оголошується скомпрометованим. Його сертифікат терміново відкликається через CRL або OCSP, усі цифрові підписи, створені після моменту інциденту, підлягають анулюванню та повторному підписанню новим ключем, а перешифрування конфіденційних даних виконується в аварійному пріоритетному порядку з одночасним знищенням старого ключа та проведенням розслідування інциденту.

Висновки

Під час виконання лабораторної роботи ми дослідили теоретичні та практичні засади безпечного керування криптографічними ключами в програмних системах, механізми захисту секретної інформації в оперативній пам'яті, вразливості реалізацій до атак за побічними каналами, а також описали систему керування ключами та проаналізували їхній життєвий цикл.

Аналізуючи способи зберігання ключової інформації, ми з'ясували, що розміщення секретів у вихідному коді, конфігураційних файлах або спільній із застосунком файловій системі створює критичні ризики витоку через системи контролю версій, дампи чи резервні копії. Надійний захист забезпечується використанням ізольованих сховищ операційних систем (Windows Credential Manager, DPAPI, macOS Keychain, Android Keystore), спеціалізованих менеджерів секретів (KMS) або апаратних криптографічних модулів (TPM, Secure Enclave, HSM). Застосування апаратної ізоляції в поєднанні з атрибутом non-exportable та стандартизованим інтерфейсом PKCS#11 дозволяє виконувати операції шифрування й цифрового підпису без завантаження закритих ключів у звичайну оперативну пам'ять хост-системи.

Дослідження захисту ключів у оперативній пам'яті та атак за побічними каналами підтвердило, що математичної стійкості криптографічного алгоритму недостатньо для гарантування безпеки системи. Відкриті ключі в RAM можуть бути скомпрометовані через механізми memory dump, crash dump або область підкачки (swap), а в мовах із автоматичним керуванням пам'яттю їх гарантоване затирання ускладнюється недетермінованістю збирача сміття та незмінністю рядків, тому для роботи із секретами слід використовувати змінні масиви байтів зі спеціальними функціями безпечного очищення або виконувати криптографічні операції безпосередньо всередині HSM. Крім того, ми переконалися, що навіть стійкий алгоритм можна скомпрометувати через витік інформації за побічними каналами — через час виконання операцій (timing attacks), особливості використання кеш-пам'яті (cache attacks), споживання електроенергії або електромагнітне випромінювання. Зокрема, використання стандартних функцій порівняння з раннім виходом для перевірки MAC чи токенів робить застосунок вразливим до атак за часом виконання, тому критично важливо застосовувати спеціалізовані функції порівняння за сталим часом (constant-time comparison) із перевірених бібліотек.

Розбір типових інцидентів порушення безпеки (потрапляння ключа до публічного Git-репозиторію, зберігання master key поруч із базою даних, логування секретів, компрометація ключа підпису) довів, що помилки конфігурування або відсутність ізоляції здатні повністю знівелювати надійність криптографічних алгоритмів. У разі компрометації недостатньо просто видалити секрет із поточного стану системи — необхідний чіткий регламент реагування, що охоплює очищення історії репозиторію, аудит журналів подій, негайне відкликання сертифікатів через CRL/OCSP та перешифрування даних.

У межах проєктної частини роботи ми описали архітектуру системи керування ключами для клієнт серверного застосунку з розподілом сховищ секретів за рівнем крити-

чності та детально проаналізували життєвий цикл ключів шифрування БД і цифрового підпису. Ми дійшли висновку, що комплексна безпека ключової інфраструктури досягається лише завдяки поєднанню апаратної ізоляції, принципу мінімально необхідних прав доступу, конвертного шифрування (Envelope Encryption) із версіонуванням ключів для безпечної планової ротації, а також процедур гарантованого затирання нулями (zeroization) після завершення їхнього життєвого циклу.

Перелік використаних джерел

Література

[1] OWASP Foundation. *Key Management Cheat Sheet.* `https://cheatsheetseries.owasp.org/cheatsheets/Key_Management_Cheat_Sheet.html`

[2] OWASP Foundation. *Cryptographic Storage Cheat Sheet.* `https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html`

[3] Microsoft. *Windows Data Protection.* `https://learn.microsoft.com/en-us/dotnet/standard/security/how-to-use-data-protection`

[4] Apple Developer. *Keychain Services.* `https://developer.apple.com/documentation/security/keychain_services`

[5] Android Developers. *Android Keystore System.* `https://developer.android.com/privacy-and-security/keystore`

[6] Microsoft. *Trusted Platform Module Technology Overview.* `https://learn.microsoft.com/en-us/windows/security/hardware-security/tpm/trusted-platform-module-overview`

[7] Apple Developer. *Protecting Keys with the Secure Enclave.* `https://developer.apple.com/documentation/security/protecting-keys-with-the-secure-enclave`

[8] Amazon Web Services. *What is AWS CloudHSM?.* `https://docs.aws.amazon.com/cloudhsm/latest/userguide/`

[9] National Institute of Standards and Technology. *FIPS 140-3: Security Requirements for Cryptographic Modules.* `https://csrc.nist.gov/pubs/fips/140-3/final`

[10] National Institute of Standards and Technology. *Guidelines for Media Sanitization (NIST SP 800-88 Rev. 2).*
https://csrc.nist.gov/pubs/sp/800/88/r2/final

[11] National Institute of Standards and Technology. *Recommendation for Key Management: Part 1 – General (NIST SP 800-57 Part 1 Rev. 5).*
https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final

[12] OASIS Open. *PKCS #11 Specification Version 3.1.* 23 July 2023.
https://docs.oasis-open.org/pkcs11/pkcs11-spec/v3.1/os/pkcs11-spec-v3.1-os.html

[13] Common Criteria Recognition Arrangement. *Common Criteria for Information Technology Security Evaluation. Part 1: Introduction and general model (CC:2022 Release 1).*
https://www.commoncriteriaportal.org/files/ccfiles/CC2022PART1R1.pdf

[14] PCI Security Standards Council. *Payment Card Industry (PCI) PIN Transaction Security (PTS) Hardware Security Module (HSM): Modular Security Requirements, Version 3.0.* June 2016.
https://listings.pcisecuritystandards.org/documents/PCI_HSM_Security_Requirements_v3_2016_final.pdf

[15] The MITRE Corporation. *CWE-316: Cleartext Storage of Sensitive Information in Memory.*
https://cwe.mitre.org/data/definitions/316.html